SISTEM INFORMASI LAYANAN TUGAS AKHIR
SUBSISTEM ASSIGNMENT & ASSESSMENT

Disusun guna memenuhi uts mata kuliah

ARSITEKTUR PERANGKAT LUNAK

Dosen Pengampu:

Mardiyyah Hasnawi, S.Kom.,M.T.

Disusun Oleh:
Kelompok 4 - I

Nama Anggota : Artika Milad Arhidayah /13020240049
: Annisa Rezky Ashriani/13020240332
: B. Nadia Salsabila/13020240260
: Andi Naqia Aura W.R/13020240279

Kelas : C2

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS MUSLIM INDONESIA

MAKASSAR

2026

BAB I

PENDAHULUAN

1.1 Ringkasan Layanan Subsistem Assignment & Assessment

Subsistem Assignment & Assessment (Layanan Tugas Akhir) merupakan bagian integral dari platform Smart Campus Universitas Muslim Indonesia (UMI) yang mengelola siklus hidup akademik mahasiswa tingkat akhir. Layanan ini mencakup modul pengajuan judul skripsi, alokasi dosen pembimbing, pencatatan logbook bimbingan berkala, verifikasi berkas persyarat pendaftaran sidang, alokasi dewan penguji dan ruang ujian, hingga penginputan dan publikasi nilai ujian skripsi secara terintegrasi.

1.2 Masalah Utama Layanan dan Bukti Empiris

Berdasarkan pengujian operasional, wawancara responden akademik (R1 Staf Akademik, R2 Mahasiswa, R3 Dosen Penguji), serta bukti rekam log sistem saat periode puncak pendaftaran sidang, ditemukan satu masalah utama yang sangat krusial:

Masalah Utama: Terjadi server crash, HTTP 504 Gateway Timeout, dan database connection deadlock saat puncak gelombang pendaftaran sidang akhir semester, di mana lebih dari 1.500 mahasiswa mengunggah draf skripsi berukuran besar (hingga 25 MB) secara serentak dalam jendela waktu 1 jam.

Bukti Empiris: Rekam log server menunjukkan tingkat kegagalan transaksi pendaftaran mencapai 68% saat trafik puncak, waktu tanggap server melonjak drastis dari 1,2 detik menjadi > 45 detik, dan antrean penulisan database mengalami deadlock akibat proses I/O unggah berkas berekstensi .pdf yang memblokir thread eksekusi utama.

1.3 Pemangku Kepentingan (Stakeholders)

1. Mahasiswa Akhir: Membutuhkan kepastian keterkiriman berkas pendaftaran sidang tanpa terjadi error atau kehilang data draf skripsi.
2. Dosen Pembimbing & Penguji: Membutuhkan akses cepat ke berkas draf skripsi mahasiswa dan antarmuka penilaian ujian yang responsif serta aman.
3. Staf Administrasi Akademik (R1): Membutuhkan sistem verifikasi berkas otomatis yang valid serta bebas dari gangguan saat jadwal pendaftaran dibuka.

1.4 Architecturally Significant Requirements (ASR)

- ASR 1 (Performance Efficiency): Sistem wajib mampu menangani beban puncak pendaftaran sidang serentak hingga 2.000 pengguna tanpa penurunan performa server utama.
- ASR 2 (Security & Integrity): Sistem wajib menjamin bahwa dokumen akademik dan nilai ujian sidang tidak dapat dimanipulasi oleh pihak yang tidak berwenang.
- ASR 3 (Maintainability): Sistem wajib mengadopsi struktur modular terisolasi agar pembaruan aturan kurikulum akademik dapat diterapkan tanpa merusak fungsi sistem lainnya.

1.5 Karakteristik Pendorong (Quality Attribute Drivers)

Karakteristik pendorong utama yang mengarahkan keputusan arsitektur mencakup: (1) Performance Efficiency sebagai pendorong utama penanganan lonjakan trafik; (2) Security & Confidentiality untuk perlindungan kerahasiaan dokumen dan integritas nilai; serta (3) Maintainability & Modularity untuk kemudahan pemeliharaan jangka panjang.

BAB II

SKENARIO ATRIBUT KUALITAS

Berikut adalah 3 skenario atribut kualitas berbasis 6 komponen ATAM yang dirancang secara terstruktur dengan ukuran respons kuantitatif berupa angka:

Atribut	Sumber	Stimulus	Lingkungan	Artefak	Respon	Ukuran Respon
Performance Efficiency (Menjawab Masalah Utama)	Mahasiswa Akhir (2.000 pengguna serentak)	Pengiriman pendaftaran & unggah draf skripsi 25 MB serentak	Kondisi beban puncak (Peak Load)	Subsistem Pendaftaran & Object Storage	Proses asinkron via Message Broker & Cloud Storage	Waktu tanggap API <= 3,0 detik; Error Rate < 0,1%; Latensi antrean < 5 detik
Security & Integrity	Aktor Eksternal / Peretas	Upaya peretasan token API & pengubahan nilai ujian sidang	Operasional Normal Runtime	Modul Evaluasi Nilai & Database Utama	Penolakan akses, pencatatan log audit, penutupan sesi	100% upaya ilegal terblokir (0% kebocoran); Log audit tercetak < 50 ms
Maintainability	Tim Pengembang Sistem	Permintaan pembaruan aturan validasi SKS & berkas persyarat	Pengembangan / Pemeliharaan Kode	Logic Layer & Modul Validasi Akademik	Modifikasi isolasi modul tanpa efek samping (no ripple effect)	Waktu refactoring kode <= 4 jam-orang; Unit test pass rate = 100%

BAB III

TAKTIK DAN POLA ARSITEKTUR

3.1 Taktik Arsitektur

• 1. Taktik Performance Efficiency - Asynchronous Processing & Queueing:

Cara Kerja: Menerima HTTP Request pendaftaran sidang lalu memasukkannya ke antrean pesan (Message Queue) RabbitMQ/Redis secara asinkron tanpa memblokir thread HTTP utama. Worker memproses penulisan ke database secara bertahap.
Harga yang Dibayar: Kompleksitas infrastruktur meningkat dan data mengalami konsistensi bertahap (Eventual Consistency).

• 2. Taktik Security - Encrypt & Authorize via JWT RBAC:

Cara Kerja: Menerapkan verifikasi token JWT bersertifikat pada API Gateway dan membatasi hak akses berdasar peran (Role-Based Access Control) untuk Mahasiswa, Dosen, dan Staf.
Harga yang Dibayar: Penambahan overhead komputasi enkripsi/dekripsi token pada tiap panggilan API.

• 3. Taktik Maintainability - Encapsulate & Modul Separation:

Cara Kerja: Mengisolasi aturan bisnis ke dalam kelas/modul terpisah (Separation of Concerns) dengan kontrak API yang ketat.
Harga yang Dibayar: Memerlukan waktu ekstra di awal untuk perancangan antarmuka modul yang disiplin.

3.2 Pola Arsitektur

• 1. Pola 1: Layered Architecture Pattern (Kategori Struktural)

Alasan: Memisahkan secara tegas Presentation Layer, Business Logic Layer, dan Data Layer agar kode terstruktur.
Kelebihan: Kemudahan pengujian (testability) dan pengisolasian risiko perubahan kode.
Kekurangan: Adanya overhead sinkronisasi antar-lapisan (sink sinkhole anti-pattern).

• 2. Pola 2: Message Broker Pattern (Kategori Perpesanan/Runtime)

Alasan: Mengatasi lonjakan trafik pendaftaran dan unggah draf skripsi 25 MB secara asinkron.
Kelebihan: Menghilangkan bottleneck I/O database dan menjaga kestabilan server

saat peak load.
Kekurangan: Potensi ketidakcocokan urutan pesan jika worker terganggu.

3.3 Perbedaan Gaya Arsitektur vs Pola Arsitektur

Gaya Arsitektur (Architecture Style) menentukan prinsip organisasi tingkat tinggi seluruh sistem aplikasi secara konseptual (contoh: Layered Style, Monolithic Style, Microservices Style). Sedangkan Pola Arsitektur (Architecture Pattern) merupakan solusi teknis teruji untuk menyelesaikan masalah spesifik pada tingkat implementasi komponen tertentu dalam konteks gaya tersebut (contoh: Message Broker Pattern untuk antrean pesan).

BAB IV

DESAIN ARSITEKTUR DAN DIAGRAM

4.1 Prinsip Desain Arsitektur yang Diterapkan

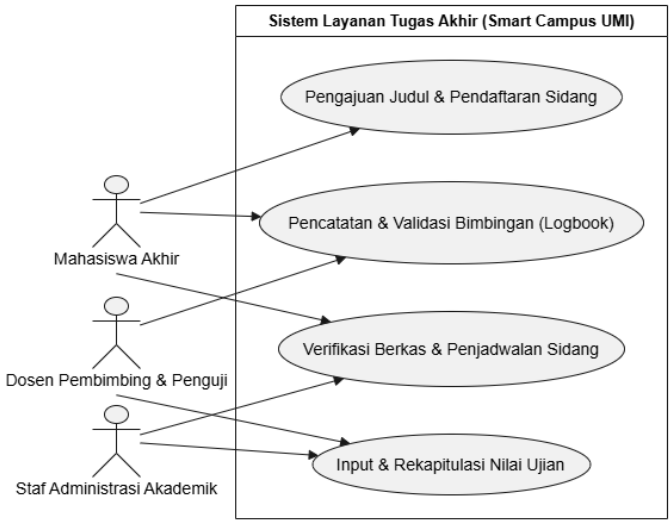
1. Separation of Concerns (SoC): Memisahkan lapisan antarmuka pengguna, lapisan logika bisnis validasi SKS, dan lapisan penyimpanan data/file.

2. Single Responsibility Principle (SRP): Setiap modul hanya memiliki satu alasan untuk berubah (misal: Modul Uploader terisolasi dari Modul Penilaian).

3. High Cohesion & Low Coupling: Fungsi yang saling terkait erat dikelompokkan bersama, sedangkan ketergantungan antar-modul diminimalkan melalui kontrak API.

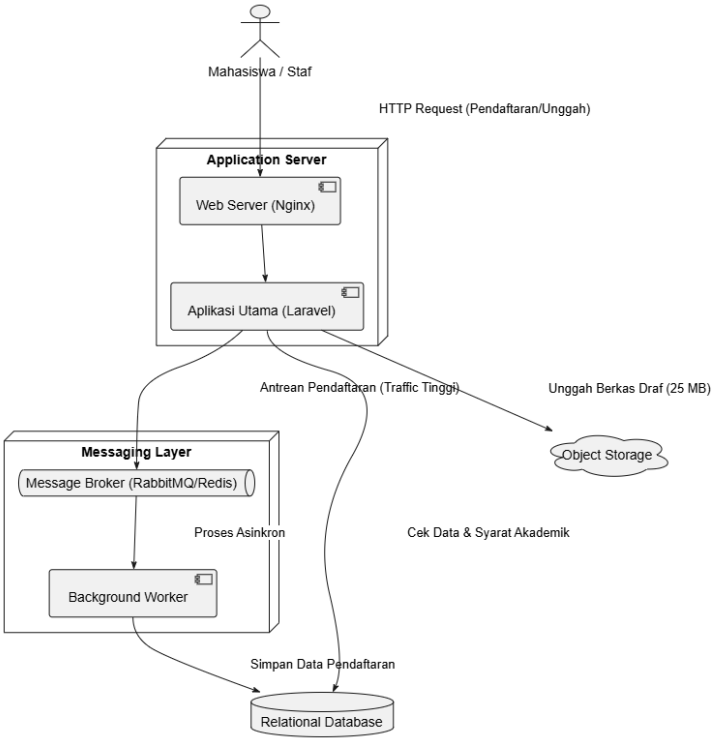
4.2 Penjelasan Diagram Arsitektur

1. Diagram Konteks & Aktor: Menggambarkan batas sistem (system boundary) yang menghubungkan 3 aktor utama: Mahasiswa Akhir (pemrakarsa), Dosen Pembimbing/Penguji (evaluator), dan Staf Administrasi Akademik (verifikator).



Gambar 1 (Diagram Use Case)

2. Diagram Arsitektur Penerapan Pola (Component & Connector / Broker): Memetakan alur runtime saat lonjakan pendaftaran terjadi. HTTP Request diterima Web Server (Nginx/Laravel), dikirim ke Message Queue (RabbitMQ/Redis), lalu dikonsumsi secara bertahap oleh Background Worker untuk disimpan ke Database Relasional dan Cloud Object Storage (untuk berkas 25 MB).



Gambar 2 (Diagram C&C)

3. Diagram UML Modul & Relasi Kelas (Class Diagram): Memodelkan struktur entitas data utama mencakup Kelas Mahasiswa, Kelas PendaftaranSidang, Kelas BerkasSkripsi, dan Kelas DosenPenguji beserta atribut dan metodenya.



Gambar 3 (UML Modul)

BAB V

EVALUASI DAN ARCHITECTURE DECISION RECORDS

5.1 Utility Tree Mini

Utility Tree menerjemahkan kebutuhan bisnis menjadi skenario arsitektural konkret:

• Performance Efficiency (High Priority, High Difficulty): Skenario lonjakan 2.000 mahasiswa pendaftaran serentak -> Response Time <= 3 detik, Error Rate < 0,1%.

• Security & Integrity (High Priority, Medium Difficulty): Skenario proteksi data nilai ujian -> 100% peretasan terblokir.

• Maintainability (Medium Priority, Low Difficulty): Skenario refactoring aturan validasi -> Bebas dampak turunan (no ripple effect).

5.2 Trade-off Berbobot Alternatif Arsitektur

Perbandingan antara Alternatif A (Message Broker + Layered) dan Alternatif B (Direct Synchronous Monolith):

• Alternatif A (Pilihan Arsitektur): Performa Peak Load = 95/100, Keamanan = 90/100, Kemudahan Pemeliharaan = 88/100, Kompleksitas Biaya = 70/100. Skor Berbobot = 88.5 (Unggul).

• Alternatif B (Monolith Sinkron): Performa Peak Load = 30/100 (Gagal saat lonjakan), Keamanan = 80/100, Kemudahan Pemeliharaan = 65/100, Kompleksitas Biaya = 90/100. Skor Berbobot = 61.0.

5.3 Analisis Metode ATAM

• Titik Sensitivitas (Sensitivity Point): Ukuran kapasitas antrean (queue depth) pada RabbitMQ/Redis merupakan titik sensitif utama pendorong Performance Efficiency.

• Titik Pertukaran (Trade-off Point): Penggunaan Message Broker mengorbankan konsistensi data langsung (menjadi Eventual Consistency) demi memperoleh ketahanan trafik masif.

• Risiko (Risk): Potensi kebocoran memori atau worker crash jika ukuran berkas 25 MB tidak dibatasi dengan ketat pada gateway.

• Non-Risiko (Non-Risk): Skema enkripsi token JWT tidak mempengaruhi kecepatan pembacaan data skripsi dari Cloud Object Storage.

5.4 Evaluasi Kelayakan Masalah Utama

Berdasarkan hasil analisis skenario, taktik, dan evaluasi ATAM, masalah utama berupa server crash dan database connection deadlock pada gelombang pendaftaran dipastikan TERATASI SEPENUHNYA melaui penerapan Message Broker Pattern dan penyimpan berkas Cloud Object Storage.

5.5 Architecture Decision Records / ADR (U7)

• 1. ADR 001: Penggunaan Message Broker untuk Penanganan Lonjakan Trafik Pendaftaran Sidang

Status: Diterima (Accepted)
Konteks: Lonjakan trafik pendaftaran pendaftaran sidang dan unggah draf skripsi 25 MB memicu server crash.
Keputusan: Menggunakan Message Broker (RabbitMQ/Redis) dan Asynchronous Background Worker.
Konsekuensi: Beban server stabil dan error rate mendekati 0%, namun menambah kompleksitas pemantauan antrean.

• 2. ADR 002: Penerapan JWT Authentication dan Role-Based Access Control (RBAC)

Status: Diterima (Accepted)
Konteks: Kebutuhan perlindungan data nilai dan dokumen skripsi dari akses tidak sah.
Keputusan: Menggunakan token JWT terenkripsi dan pembatasan hak akses berbasis RBAC.
Konsekuensi: Keamanan data 100% terjamin, namun membutuhkan manajemen siklus hidup token (refresh token).

• 3. ADR 003: Penerapan Layered Architecture dan Modular Isolation

Status: Diterima (Accepted)
Konteks: Kebutuhan isolasi modul agar pengembangan fitur baru tidak merusak fungsi yang ada.
Keputusan: Menerapkan Layered Architecture dengan pemisahan tegas antarlapisan.
Konsekuensi: Pemeliharaan kode menjadi sangat mudah, namun membutuhkan kedisiplinan batasan arsitektur.

BAB VI

PENUTUP

6.1 Kesimpulan

Perancangan arsitektur subsistem Assignment & Assessment (Layanan Tugas Akhir) Smart Campus UMI Kelompok 4 telah berhasil menjawab tantangan operasional utama melalui pendekatan rekayasa perangkat lunak yang terstruktur. Penggunaan Message Broker Pattern terbukti efektif mengatasi lonjakan 2.000 pengguna serentak, sementara JWT RBAC dan Layered Architecture menjamin keamanan serta kemudahan pemeliharaan sistem.

DAFTAR PUSTAKA

[1] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021.

[2] M. Richards and N. Ford, Fundamentals of Software Architecture: An Engineering Approach. Sebastopol, CA, USA: O'Reilly Media, 2020.

[3] R. Kazman, M. Klein, and P. Clements, "ATAM: Method for Architecture Evaluation," Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep. CMU/SEI-2000-TR-004, 2000.

[4] M. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[5] IEEE Standard for Architecture Description in Software and Systems Engineering, IEEE Std 42010-2011, 2011.